ML-Driven Trading on Kalshi BTC Binary Markets

Machine Learning Final Project

Todd Tan Aldrich Wang and Arush Guliani

The University of Chicago

March 12, 2026

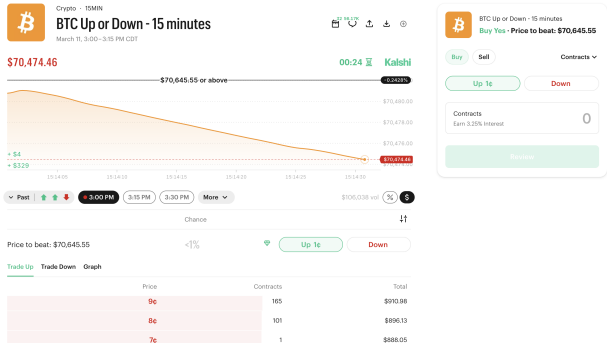
Problem Statement

The Exchange: Kalshi

- CFTC-regulated prediction market exchange
- Binary contracts: pays \$1 if event occurs, \$0 otherwise

BTC Up or Down — 15 Min

- New contract every 15 min: "Will BTC be $\geq$ \$X at expiry?"
- YES/NO sides trade as probabilities (cents); live bid/ask order book
- Settles at \$1 (YES) or \$0 (NO)



Data Sources

Kalshi API (REST & WebSocket)

- Top-of-book YES/NO: bid/ask price & size
- Trade stats: count, volume, VWAP (last 60s)
- Quote-change event count per minute
- Frequency: **20-second snapshots**, real-time

Coinbase API

- BTC/USD spot price, **every 20 seconds**, real-time

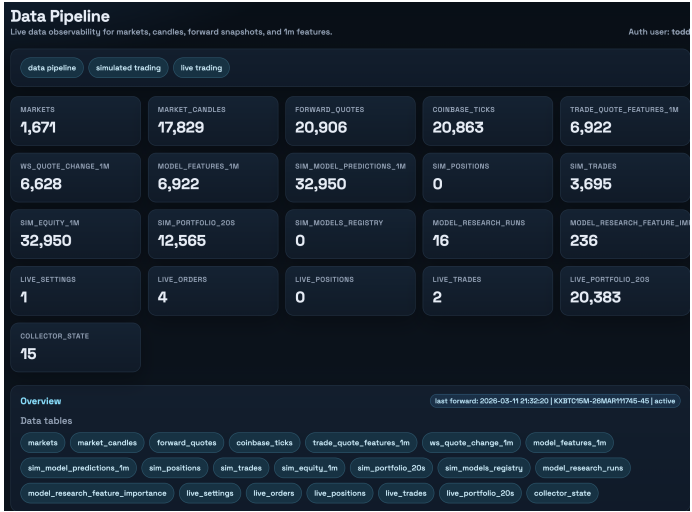
Collection & Storage

- Live rerun window: **Mar 7 02:11 – Mar 12 06:04 UTC**
- Current presentation refresh uses **7,429 eligible 1-min rows**
- MySQL on AWS RDS; 20s snapshots aggregated to 1-min bars
- EC2 workers: `forward_collector`, `coinbase_collector`, `feature_1m_collector`

Key Tables

- `forward_quotes` — 20s top-of-book
- `coinbase_ticks` — 20s BTC spot
- `model_features_1m` — 1-min ML feature table

Data Pipeline Dashboard



Feature Set

Order Book & Pricing

- `yes_bid/ask_dollars` — top-of-book prices
- `spread_dollars`, `mid_dollars`
- `orderbook_imbalance` — $(bid_sz - ask_sz)/(bid_sz + ask_sz)$

Contract Returns

- `mid_ret_1m`, `mid_ret_3m`

Trade Activity (last 60s)

- `trades_last_60s_count` — trade count
- `trades_last_60s_volume_fp` — dollar volume
- `trades_last_60s_vwap_yes` — VWAP
- `quote_change_events_60s` — book update frequency

Time to Expiry

- `time_to_expiry_sec`

Coinbase BTC Spot

- `cb_ret_1m`, `cb_ret_3m`, `cb_ret_5m`
- `cb_vol_5m` — 5-min realized volatility

Moneyness & Interactions

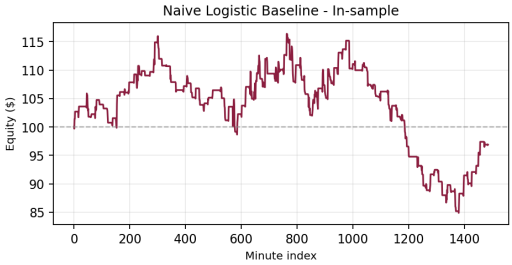
- `moneyness_abs/pct` — spot vs. strike
- `moneyness_vol_scaled` — $\div$ BTC vol
- `moneyness_x_sqrt_tte` — $\times\sqrt{TTE}$
- `moneyness_x_imbalance` — $\times$ imbalance
- `moneyness_x_qchg` — $\times$ quote changes

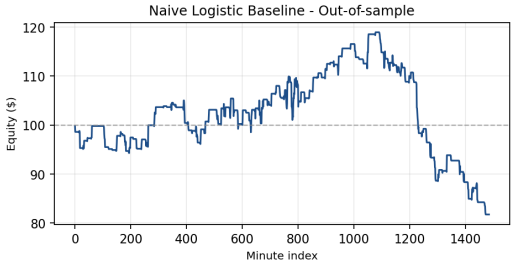
Model Baseline — Naive Logistic Regression

Label: $y_t = \mathbf{1}[\text{bid}_{t+1} > \text{ask}_t]$ **Features (13):** `bid, ask, spread, mid, imbalance, mid_ret_1m/3m, cb_ret_1m/3m, trades_count, quote_chg, tte, moneyness_pct`

Train (UTC)	Mar 7 02:11 – Mar 10 04:31	4,457 rows	Train
Validation (UTC)	Mar 10 04:32 – Mar 11 05:17	1,486 rows	In-sample holdout
OOS (UTC)	Mar 11 05:18 – Mar 12 06:04	1,486 rows	Forward test

Buy YES: $p_{\text{yes}} \geq 0.55$ & edge $> \$0.01$ **Sell YES:** $p_{\text{yes}} < 0.55$ or edge $\leq \$0.01$ (or TTE ≤ 60s)
Qty: 1–10 contracts (scaled by edge) **OOS PnL:** -**$18.26** / 135 trades / 45.2% win rate





Model #1 — Logistic Regression with L1 and Interactions

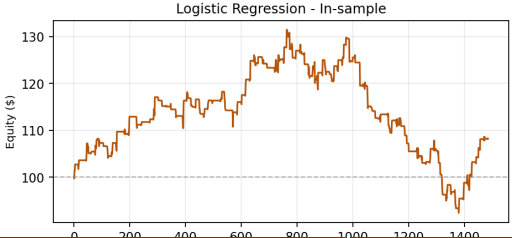
Label: $y_t = \mathbf{1}[\text{bid}_{t+1} > \text{ask}_t]$ **Features (22):** all baseline features + `cb_ret_5m`, `cb_vol_5m`, `moneyness_abs`, `moneyness_vol_scaled`, `moneyness_x_sqrt_tte`, `moneyness_x_imbalance`, `moneyness_x_trades_volume_fp`, `trades_vwap`

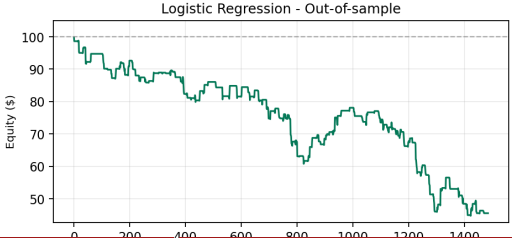
Regularization: L1 ($\lambda = 2.0$)

Train (UTC)	Mar 7 02:11 – Mar 10 04:31	4,457 rows	Train
Validation (UTC)	Mar 10 04:32 – Mar 11 05:17	1,486 rows	In-sample holdout
OOS (UTC)	Mar 11 05:18 – Mar 12 06:04	1,486 rows	Forward test

OOS PnL: -$54.49 ($100→$45.51) / 147 trades / 43.5% win rate

Comment: Validation improved (+$8.20), but OOS stayed sharply negative; linear complexity still does not hold up live





Model #2 — Gradient Boosting

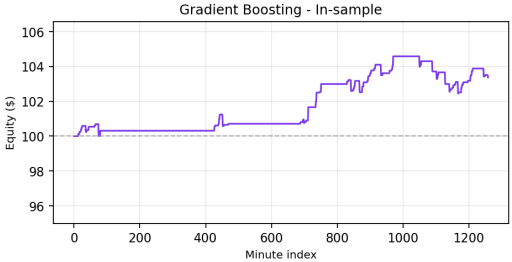
Label: $y_t = \mathbf{1}$[final YES bid $\geq$ \$0.90] (expiry-resolution) **Features (39):** all 22 production features + depth/pressure, flow context, return interactions, strike carry

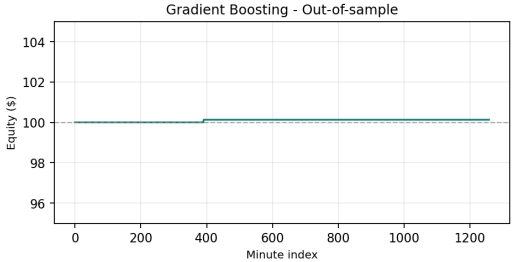
Params: `learning_rate=0.03, max_depth=4, max_iter=250, min_samples_leaf=25 l2=0.01, max_leaf_nodes=63`

Train (UTC)	Mar 7 02:11 – Mar 10 05:33	3,776 rows	60% split
Validation (UTC)	Mar 10 05:34 – Mar 11 05:46	1,259 rows	In-sample holdout
OOS (UTC)	Mar 11 05:47 – Mar 12 05:58	1,259 rows	Forward test

Buy: $p_{\text{yes}} \geq 0.60$ & edge $\geq$ \$0.20 **Exit:** take-profit at entry +\$0.05, else final YES bid **Size:** 1

OOS PnL: +\$0.13 / 1 trade / 100.0% win rate **OOS AP:** 0.820, **AUC:** 0.837





Model #3 — Random Forest

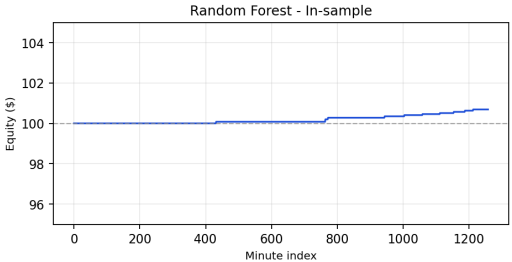
Label: $y_t = \mathbf{1}$[final YES bid $\geq$ \$0.90] (expiry-resolution) **Features (39):** all 22 production features + depth/pressure, flow context, return interactions, strike carry

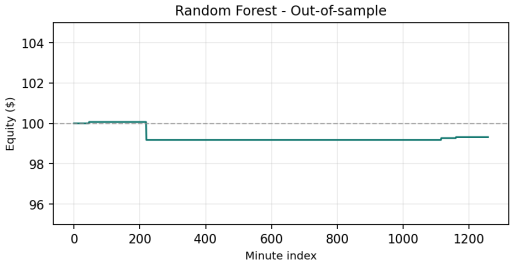
Params: `n_estimators=500, max_depth=10, min_samples_leaf=4, min_samples_split=12, max_features=sqrt, class_weight=balanced_subsample`

Train (UTC)	Mar 7 02:11 – Mar 10 05:33	3,776 rows	60% split
Validation (UTC)	Mar 10 05:34 – Mar 11 05:46	1,259 rows	In-sample holdout
OOS (UTC)	Mar 11 05:47 – Mar 12 05:58	1,259 rows	Forward test

Buy: $p_{\text{yes}} \geq 0.90$ & edge $\geq$ \$0.08 **Exit:** take-profit at entry +\$0.05, else final YES bid **Size:** 1

OOS PnL: -\$0.68 / 4 trades / 75.0% win rate **OOS AP:** 0.846, **AUC:** 0.865





Model #4 — Bagging Trees

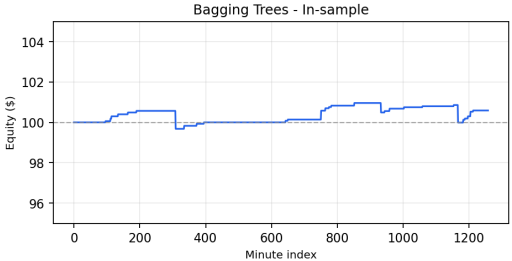
Label: $y_t = \mathbf{1}$[final YES bid $\geq$ \$0.90] (expiry-resolution) **Features (22):** same production features as logistic / random forest (order book, returns, moneyness, Coinbase)

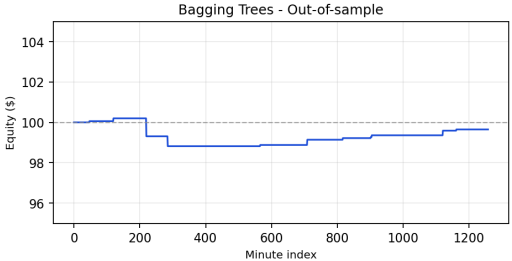
Params: `BaggingClassifier: n_estimators=100, base DecisionTreeClassifier max_depth=6, min_samples_leaf=8, min_samples_split=16, max_samples=0.8, max_features=0.8`

Train (UTC)	Mar 7 02:11 – Mar 10 05:33	3,776 rows	60% split
Validation (UTC)	Mar 10 05:34 – Mar 11 05:46	1,259 rows	In-sample holdout
OOS (UTC)	Mar 11 05:47 – Mar 12 05:58	1,259 rows	Forward test

Buy: $p_{\text{yes}} \geq 0.55$ & edge $\geq$ \$0.08 **Exit:** take-profit at entry +\$0.05, else final YES bid **Size:** 1

OOS PnL: -\$0.35 / 11 trades / 81.8% win rate **OOS AP:** 0.858, **AUC:** 0.875





Model #5 — AdaBoost (Boosted Trees)

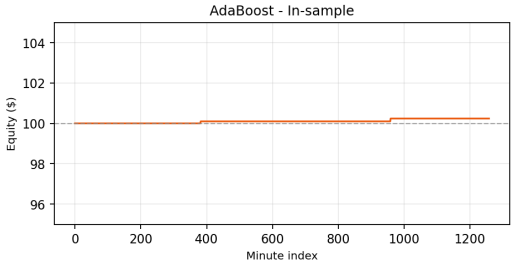
Label: $y_t = \mathbf{1}$[final YES bid $\geq$ \$0.90] (expiry-resolution) **Features (22):** same production features as logistic / random forest (order book, returns, moneyness, Coinbase)

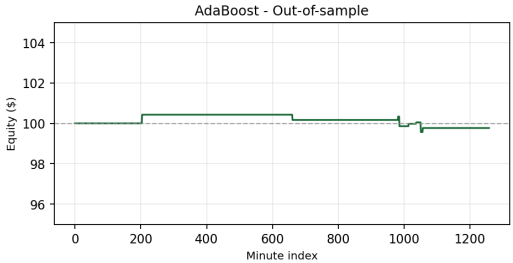
Params: `AdaBoostClassifier`: `n_estimators=100, learning_rate=0.1`, base `DecisionTreeClassifier` `max_depth=2, min_samples_leaf=4, min_samples_split=8`

Train (UTC)	Mar 7 02:11 – Mar 10 05:33	3,776 rows	60% split
Validation (UTC)	Mar 10 05:34 – Mar 11 05:46	1,259 rows	In-sample holdout
OOS (UTC)	Mar 11 05:47 – Mar 12 05:58	1,259 rows	Forward test

Buy: $p_{\text{yes}} \geq 0.55$ & edge $\geq$ \$0.00 **Exit:** take-profit at entry +\$0.05, else final YES bid **Size:** 1

OOS PnL: -\$0.23 / 8 trades / 62.5% win rate **OOS AP:** 0.848, **AUC:** 0.871





Conclusion

What we did

- Built an end-to-end pipeline: Kalshi + Coinbase data, 20s→1-min features, MySQL, shared research and live trading engine
- Re-ran the benchmark and Models 1–5 on live data from Mar 7 02:11 through Mar 12 06:04 UTC, then regenerated every slide plot from that refresh
- Expiry-model threshold tuning now requires at least 10 validation trades when possible, with fallback to the unconstrained search if no rule meets that floor

Results (in-sample vs out-of-sample)

- Linear models stayed negative OOS: benchmark -**$18.26** and L1 logistic -**$54.49**, despite modest ranking lift (AUC ∼0.66)
- Expiry-resolution tree models kept strong ranking quality (test AP/AUC roughly 0.82–0.88), but realized tradable edge was sparse on the refreshed OOS slice
- GBM finished slightly positive at +**$0.13** on 1 OOS trade; RF lost -**$0.68** on 4 trades; Bagging traded **11** times for -**$0.35**; AdaBoost lost -**$0.23** on 8 trades

Takeaways

- Predictive ranking alone is not enough; threshold selection and execution assumptions dominate realized PnL once we demand live, non-synthetic trades

Next Steps

Models

- Expand to longer walk-forward windows and retune thresholds with a minimum-trade, transaction-cost-aware objective so inactivity does not dominate model selection
- Use feature importance (e.g., Shapley) to shrink the 39-feature tree set and test whether we can preserve AP/AUC while improving trade density

Execution & risk

- Harden live engine: slippage and partial fills, expiry-boundary rules, quote-age filters, and drawdown / position limits

Monitoring

- Add calibration drift, threshold stability, and per-model trade-density panels so we can see when strong AP/AUC is failing to convert into executable edge

Thank You

Questions & Discussion